

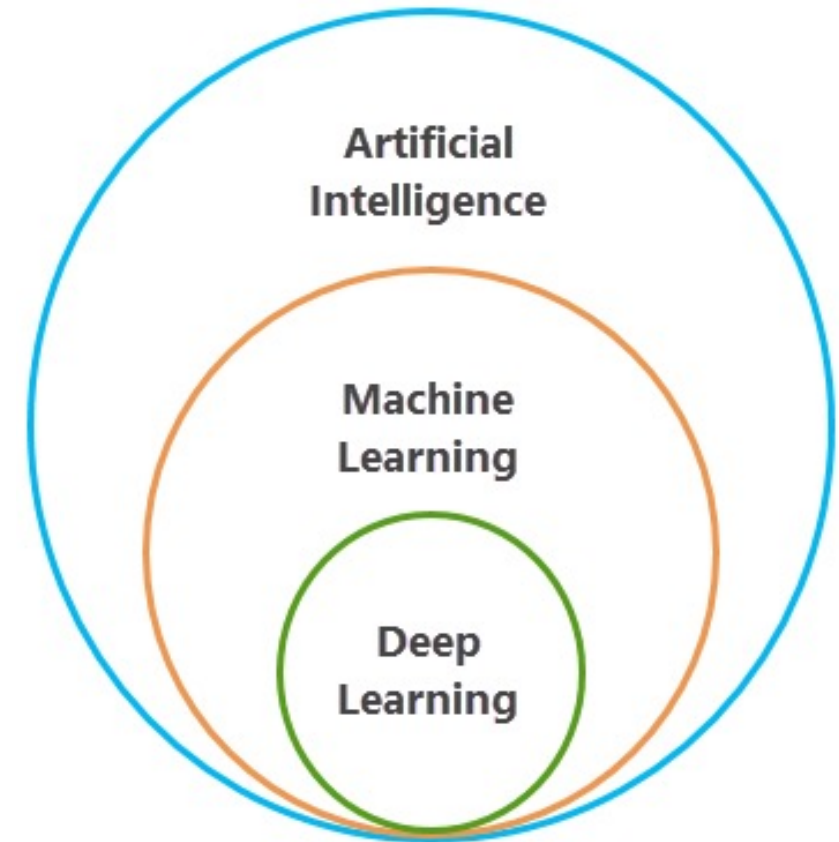
Machine Learning

Deep Learning

Dr. Ahmed Aljarah

Deep Learning

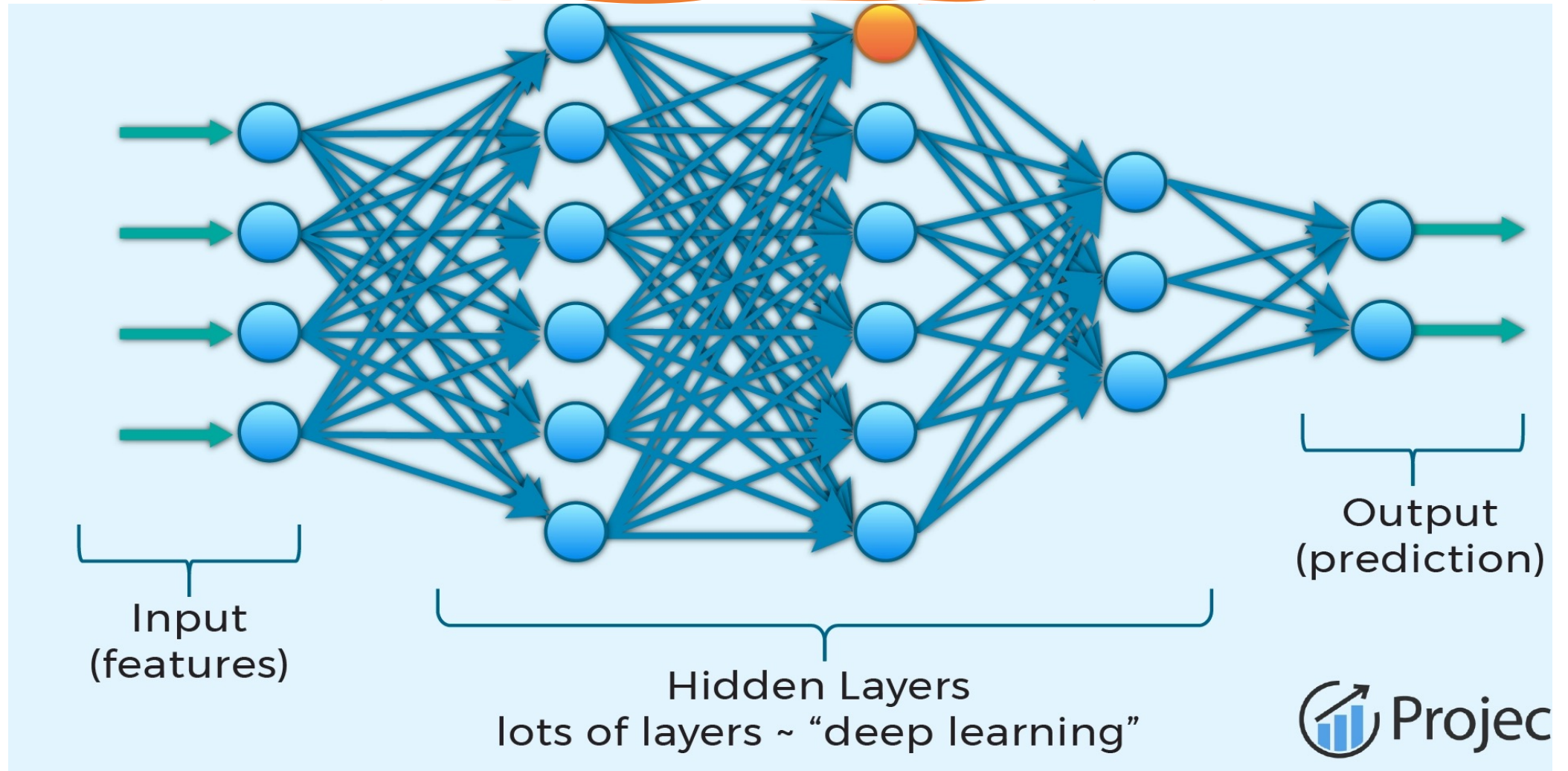
- Deep learning is a subset of machine learning based on neural networks.
- Deep learning is transforming how machines understand, learn, and interact with complex data.
- It's Inspired by the neural networks of the human brain.
- Deep learning enables computers to automatically identify patterns and make intelligent decisions using large amounts of unstructured data.



Deep Learning

- A neural network consists of interconnected neurons organized into layers.
- Deep learning uses deep neural networks with multiple hidden layers.
- More hidden layers allow the model to learn complex patterns.
- Data flows from the input layer through hidden layers to the output layer.
- Each neuron performs mathematical and nonlinear transformations on the data.
- Hidden layers progressively extract important features from the input data.

Deep Learning



Machine Learning Vs. Deep Learning



Aspect	Machine Learning	Deep Learning
Data	Small/medium datasets	Large datasets
Complexity	Simple tasks	Complex tasks
Features	Manual extraction	Automatic extraction
Training	Faster	Slower
Model	Less complex	Highly complex
Interpretability	Easier to explain	Harder to interpret
Hardware	CPU	GPU/high computing power
Use Cases	Spam detection, recommendations	Image recognition, speech recognition

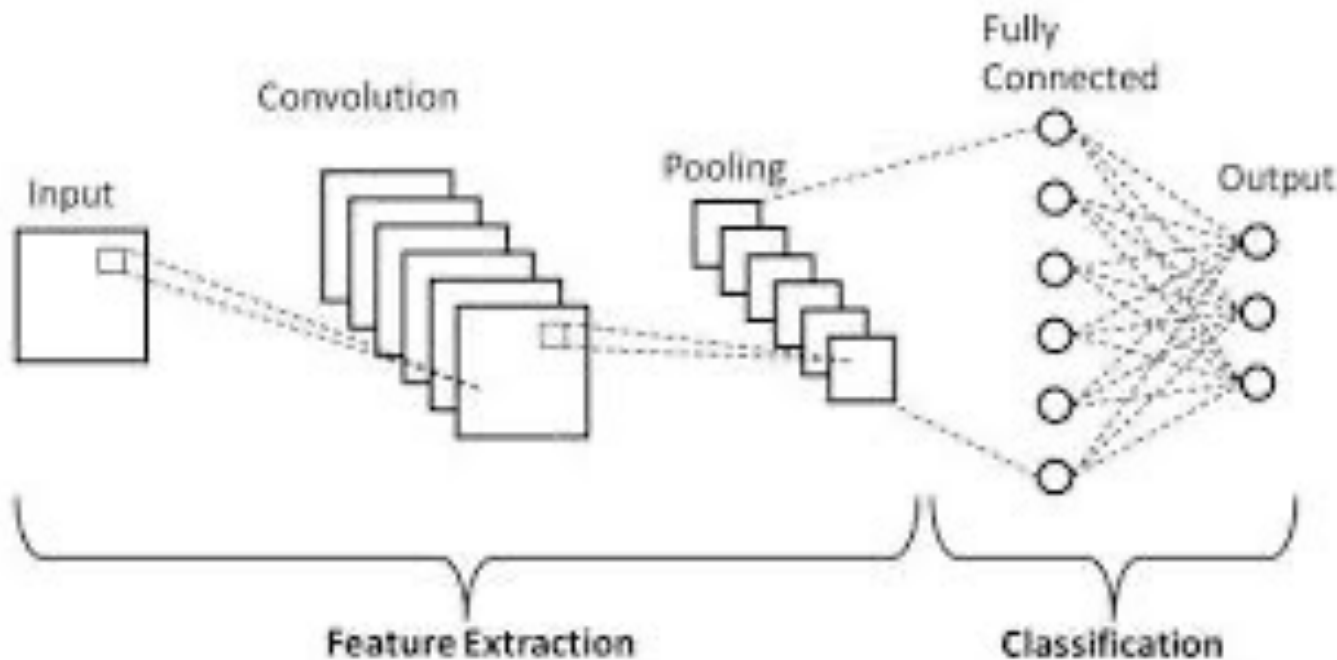
Convolutional Neural Network (CNN)

- CNN is deep learning model designed to process grid-like data such as images.
- CNN is widely used in computer vision applications.
- It automatically detect and learn important features from visual data.
- CNN uses convolutional layers to identify patterns such as edges, shapes, and textures.
- Multiple layers enable CNN to learn complex image features.
- CNN is commonly used for image classification, object detection, and facial recognition.

CNN Components

- Convolutional Layers
 - Apply filters (kernels) to input images and to detect features.
 - Identify edges, textures, shapes, and complex patterns.
 - Preserve spatial relationships between pixels.
- Activation Functions
 - Introduce nonlinearity into the network.
 - Common functions include ReLU and Tanh.
- Pooling Layers
 - Reduce the spatial dimensions of the data.
 - Lower computational complexity and number of parameters.
 - Max pooling selects the maximum value from neighboring pixels
- Fully Connected Layers
 - Use learned features to make final predictions.
 - Connect every neuron from one layer to the next.
 - Commonly used for classification tasks.

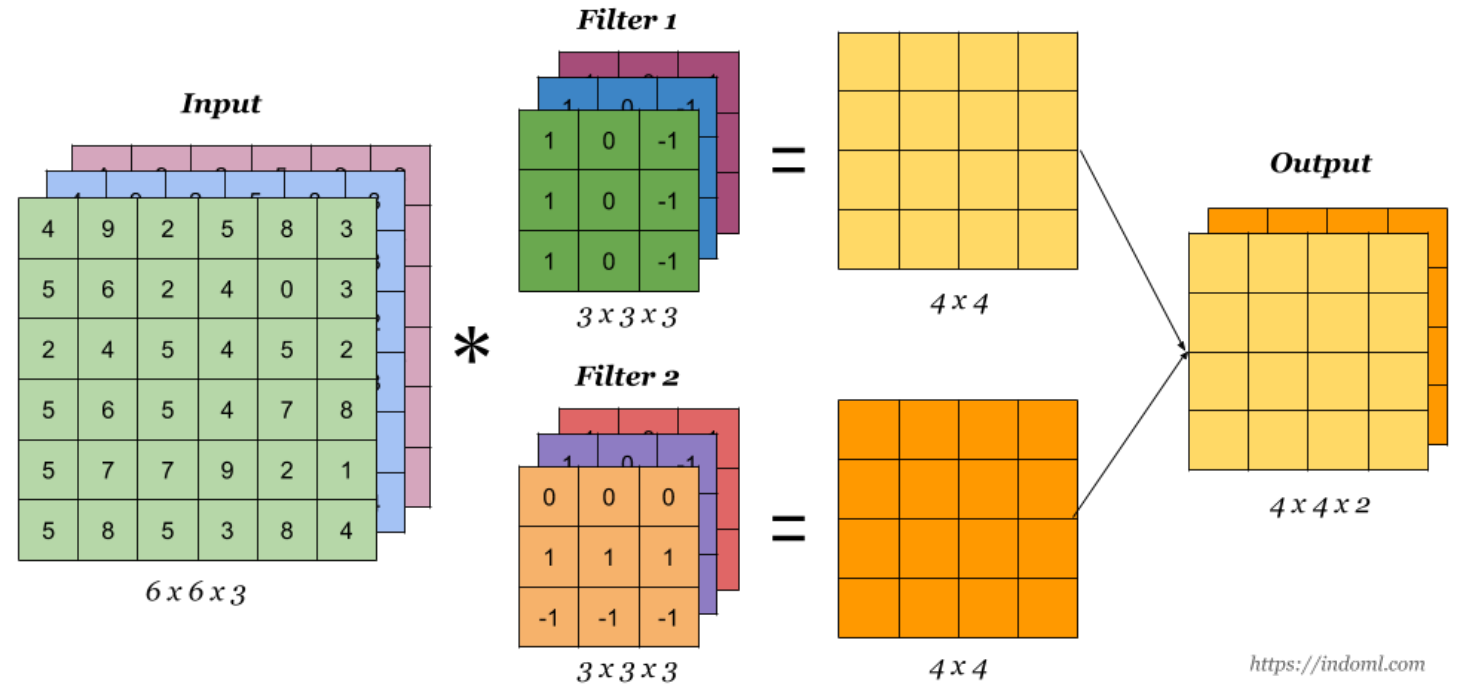
CNN Architecture



- Each layer transforms the input data into another representation using differentiable functions.
- Example input image size: $32 \times 32 \times 3$
 - 32 = width
 - 32 = height
 - 3 = RGB color channels

1. Input Layer

- Receives the image data.
- Passes the image to the network for processing.
- Input is represented as a 3D volume (width $\times$ height $\times$ depth).
- Stores pixel values while preserving spatial structure.
- Example: $6 \times 6 \times 3$ RGB image.

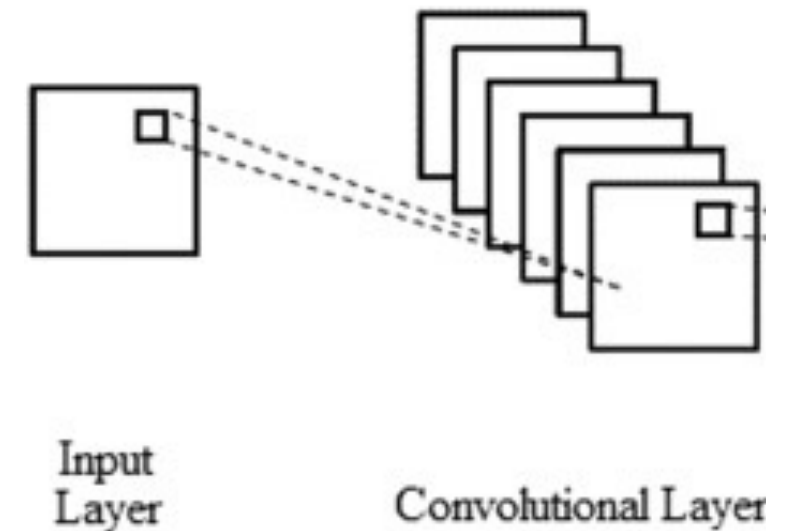


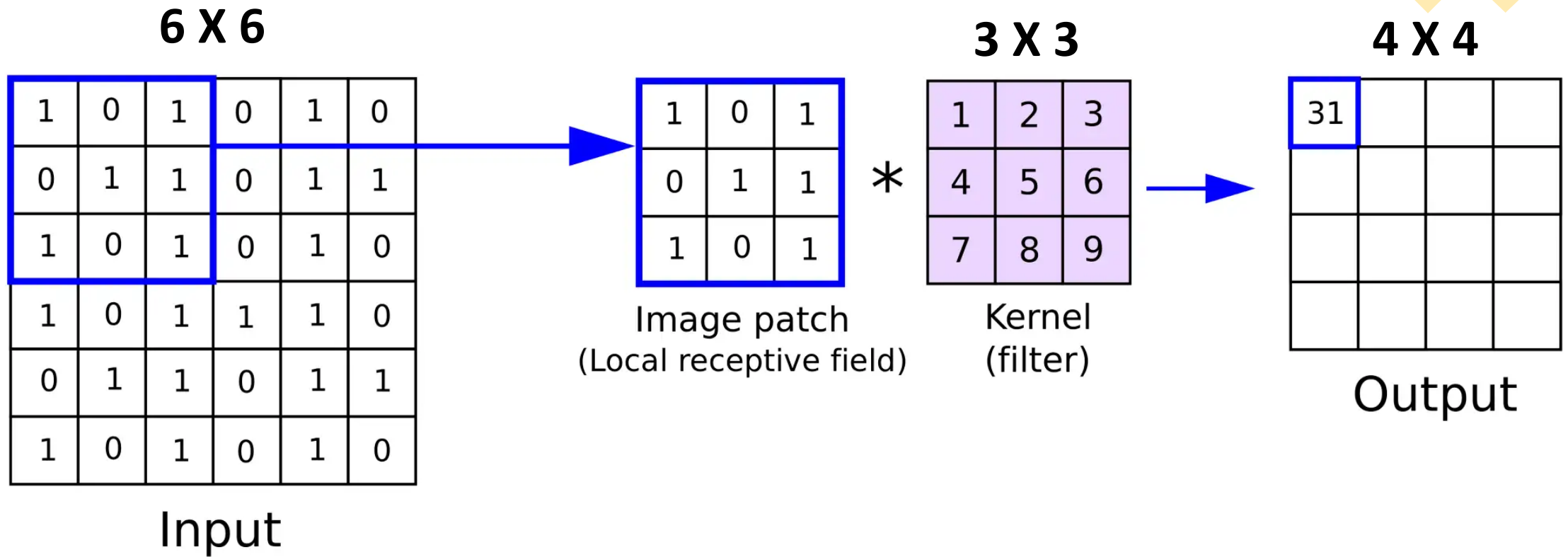
2. Convolutional Layer

- Extracts important features from the input image.
- Applies learnable filters (kernels) across the image.
- Filters detect patterns such as:
 - Edges
 - Textures
 - Shapes
- Common filter sizes: 2×2 , 3×3 and 5×5
- Produces feature maps.

Example:

Using 6 filters gives an output volume of $32 \times 32 \times 3 \times 6$.





$$(N \times N) * (F \times F) = (N - F + 1) \times (N - F + 1)$$

Convolutional layer Concepts



DEPTH



STRIDE

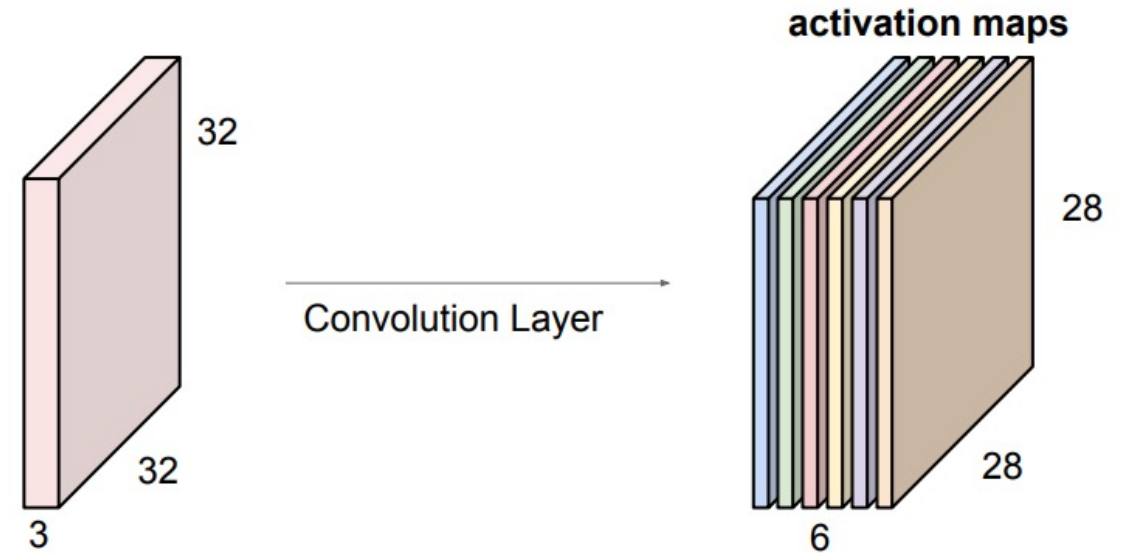


PADDING

Depth

- Depth refers to the number of filters used in a convolutional layer.
- Each filter learns to detect a different feature or pattern from the image.
- More filters produce more feature maps.
- Increasing depth allows the CNN to detect more complex patterns such as edges, textures, and shapes.

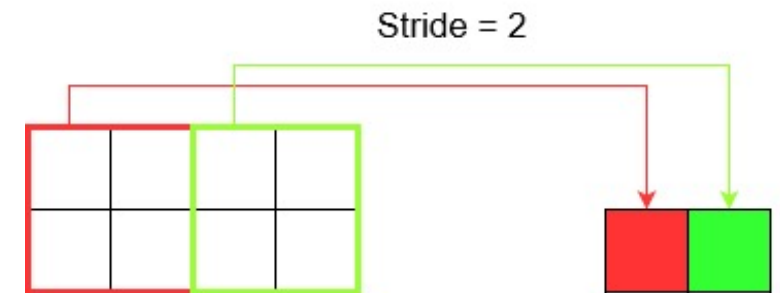
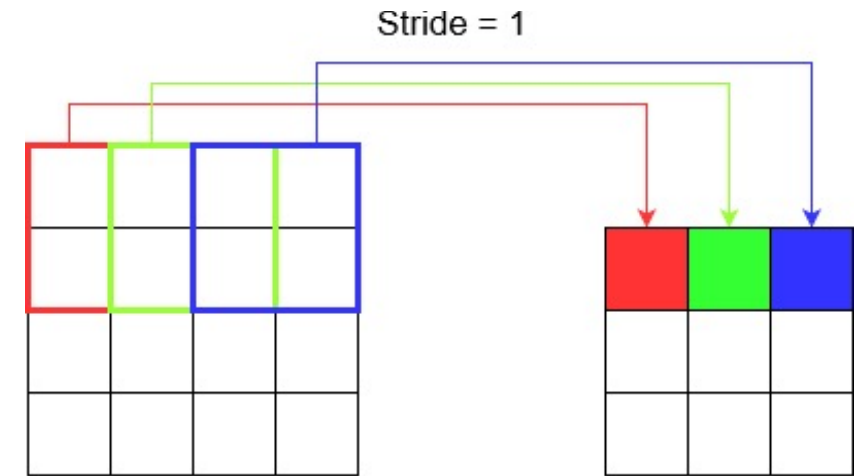
For example, if we had 6 5x5 filters, we'll get 6 separate activation maps:



We stack these up to get a “new image” of size 28x28x6!

Stride

- Stride is the number of pixels the filter moves across the image at each step.
- It controls how the filter scans the input image.
- **Common cases:**
- **Stride = 1** → filter moves one pixel at a time.
- **Stride = 2** → filter jumps two pixels at a time, producing a smaller output.
- Larger strides reduce the spatial dimensions of the output feature map.
- Smaller strides preserve more image details.



Padding

- Padding adds extra borders of zeros around the input image before convolution.
- It helps control the output size after convolution.
- Padding prevents loss of information at the image edges.

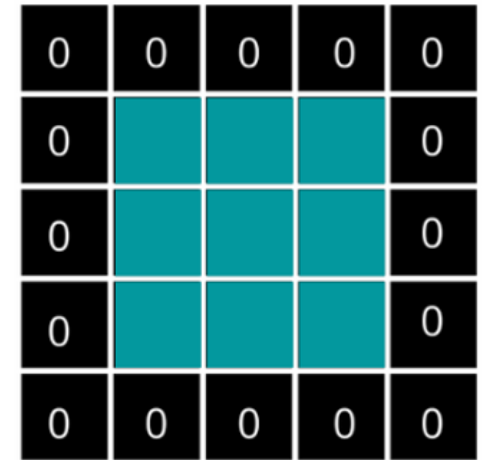
Benefits of padding:

- Preserves spatial dimensions.
- Allows filters to process edge pixels more effectively.
- Helps maintain important image information.
- Without padding, output size becomes smaller after convolution.
- With padding, the output size can remain the same as the input size.



Input Image

Applying padding
of 1 on 3x3

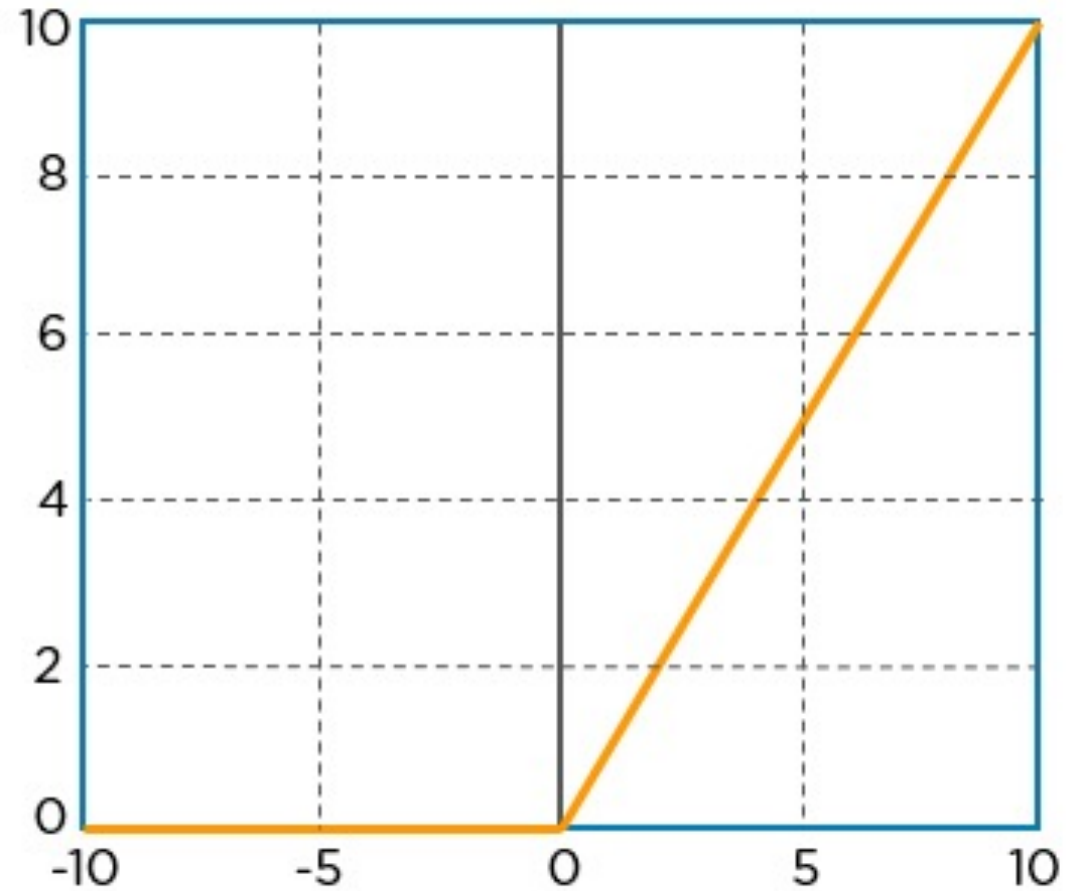


Padded Image

Input		Kernel		Output																																													
<table border="1" style="border-collapse: collapse; text-align: center;"><tr><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>0</td><td>1</td><td>2</td><td>0</td></tr><tr><td>0</td><td>3</td><td>4</td><td>5</td><td>0</td></tr><tr><td>0</td><td>6</td><td>7</td><td>8</td><td>0</td></tr><tr><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td></tr></table>	0	0	0	0	0	0	0	1	2	0	0	3	4	5	0	0	6	7	8	0	0	0	0	0	0	$*$	<table border="1" style="border-collapse: collapse; text-align: center;"><tr><td>0</td><td>1</td></tr><tr><td>2</td><td>3</td></tr></table>	0	1	2	3	$=$	<table border="1" style="border-collapse: collapse; text-align: center;"><tr><td>0</td><td>3</td><td>8</td><td>4</td></tr><tr><td>9</td><td>19</td><td>25</td><td>10</td></tr><tr><td>21</td><td>37</td><td>43</td><td>16</td></tr><tr><td>6</td><td>7</td><td>8</td><td>0</td></tr></table>	0	3	8	4	9	19	25	10	21	37	43	16	6	7	8	0
0	0	0	0	0																																													
0	0	1	2	0																																													
0	3	4	5	0																																													
0	6	7	8	0																																													
0	0	0	0	0																																													
0	1																																																
2	3																																																
0	3	8	4																																														
9	19	25	10																																														
21	37	43	16																																														
6	7	8	0																																														

3. Activation Layer

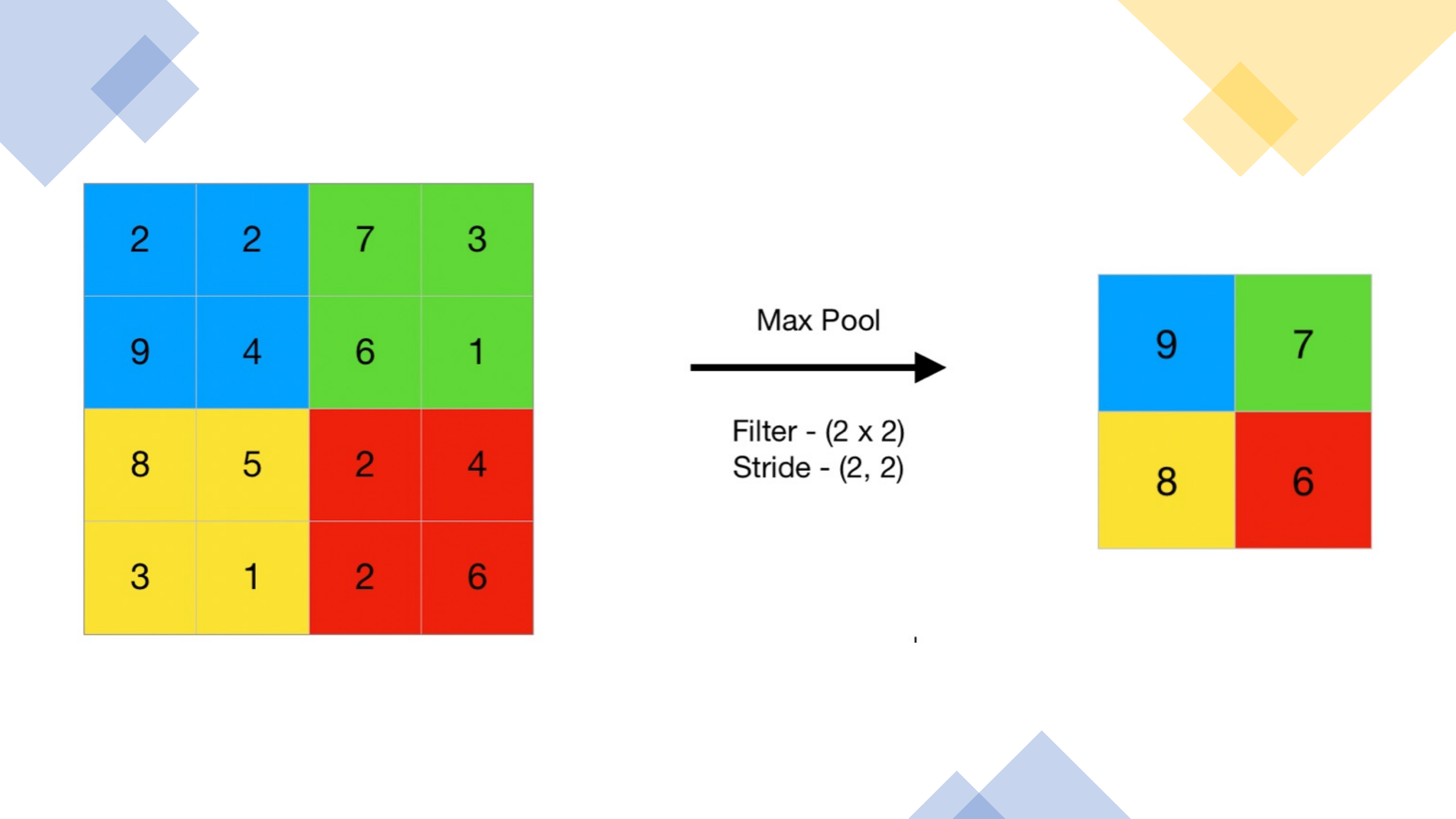
- Introduces nonlinearity into the network.
- Helps the model learn complex relationships.
- Applied element-wise to the feature maps.
- Common activation functions:
 - ReLU
 - Tanh
- Output dimensions remain unchanged.



$$R(z) = \max(0, z)$$

4. Pooling Layer

- Reduces the spatial dimensions of feature maps.
- Decreases computation and memory usage.
- Helps prevent overfitting.
- Usually placed after convolutional layers.
- Common pooling methods:
 - Max Pooling
 - Average Pooling
- Reduces width and height while keeping depth unchanged.



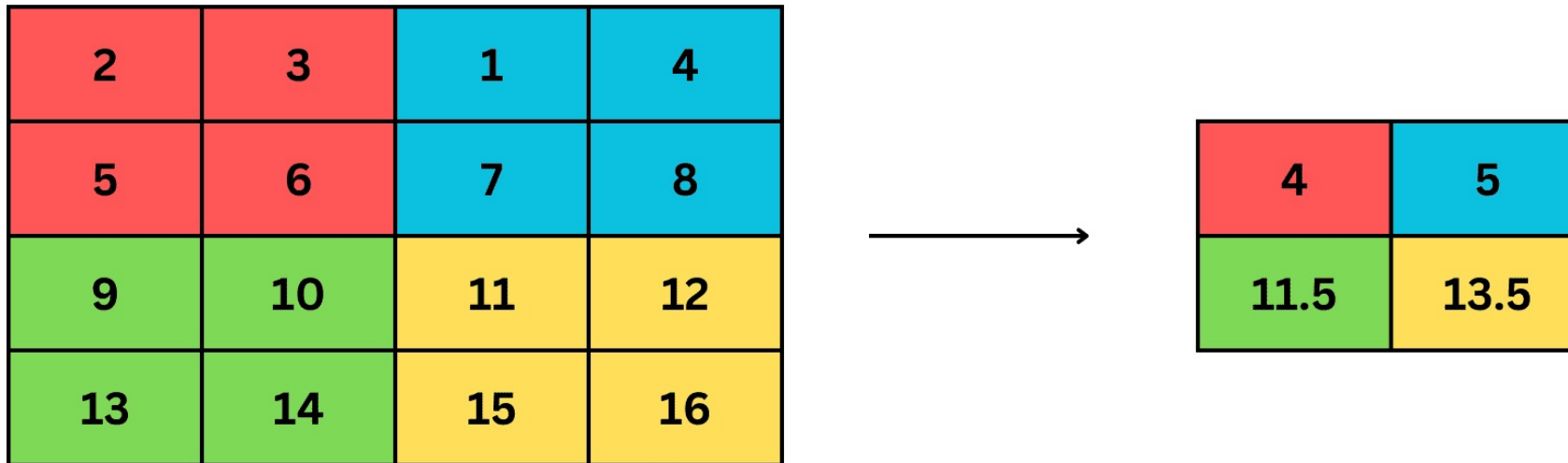
2	2	7	3
9	4	6	1
8	5	2	4
3	1	2	6

Max Pool
→

Filter - (2 x 2)
Stride - (2, 2)

9	7
8	6

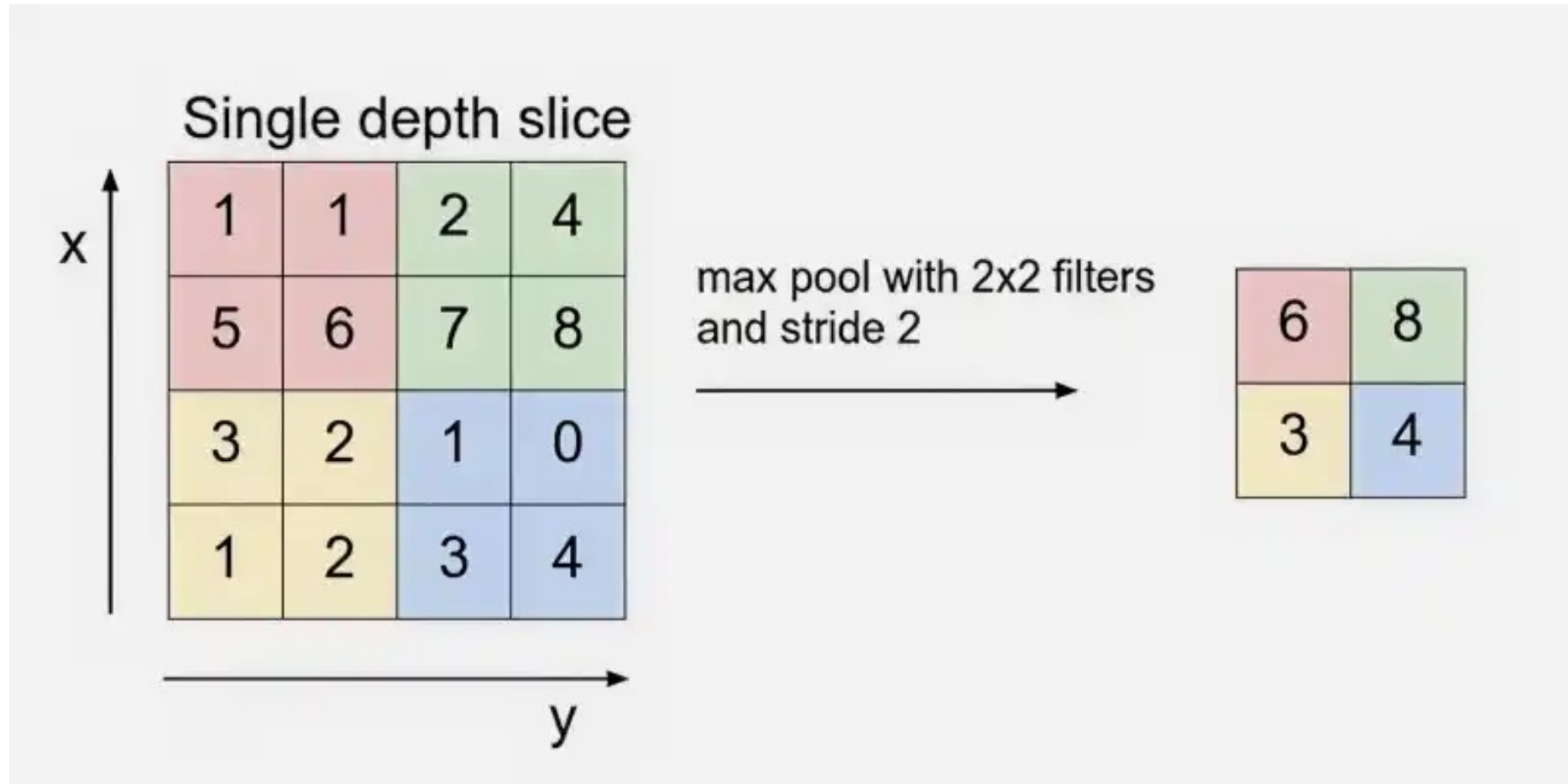
Average Pooling



Take average of all values in the window

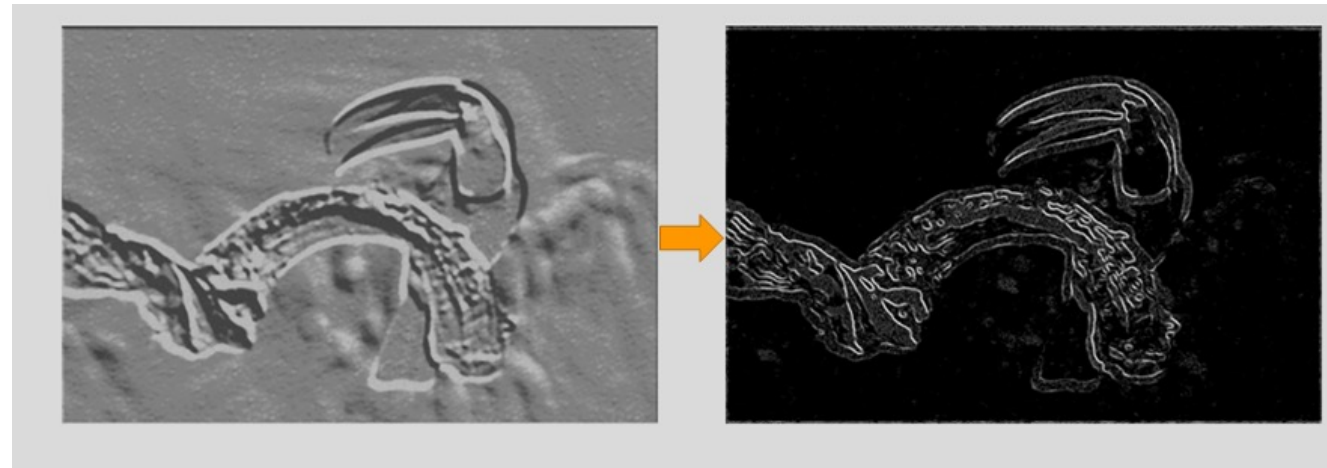
Example:

- Using 2×2 max pooling with stride 2 reduces the volume from $4 \times 4 \times 12$ to $2 \times 2 \times 12$.



Max pooling

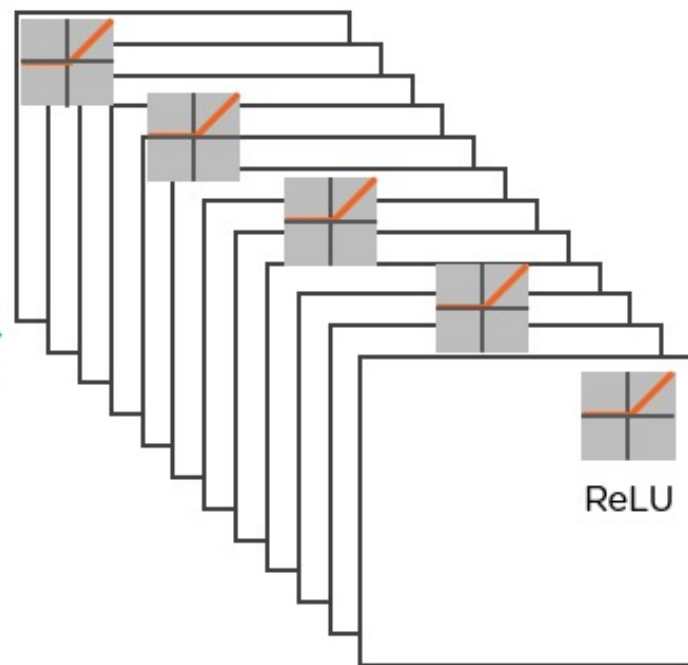
- Max pooling keeps the strongest feature value from a small region.
- It reduces feature map size while preserving important information.
- Helps the CNN recognize objects even when they slightly shift in the image (translation invariance).



1	1	1	0	0
0	1	1	1	0
0	0	1	1	1
0	0	1	1	0
0	1	1	0	0

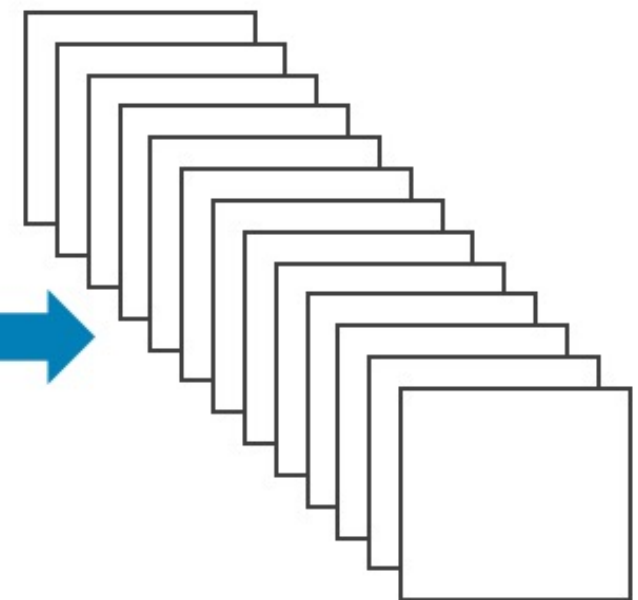
Input Image

Convolution



Convolution Layer

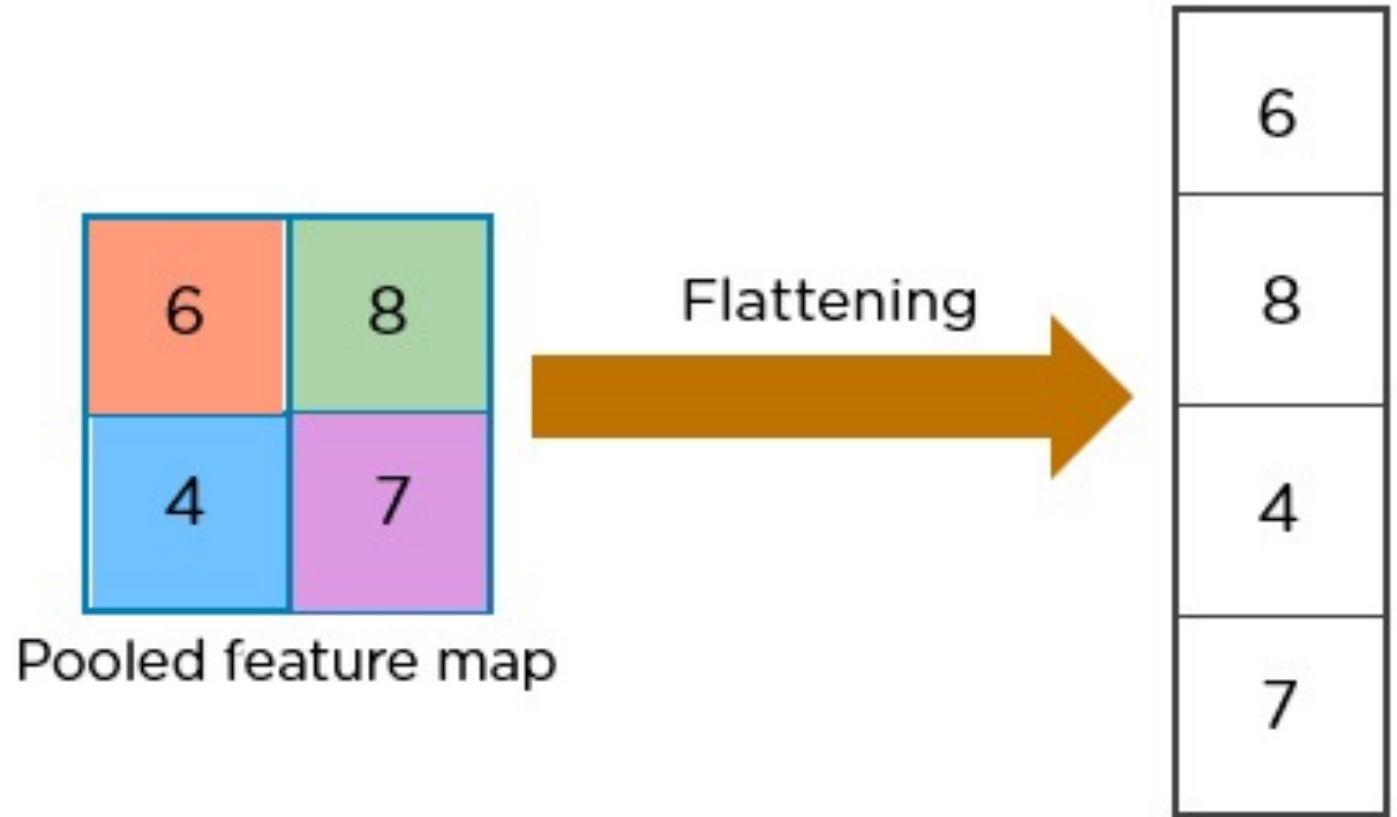
Pooling



Pooling Layer

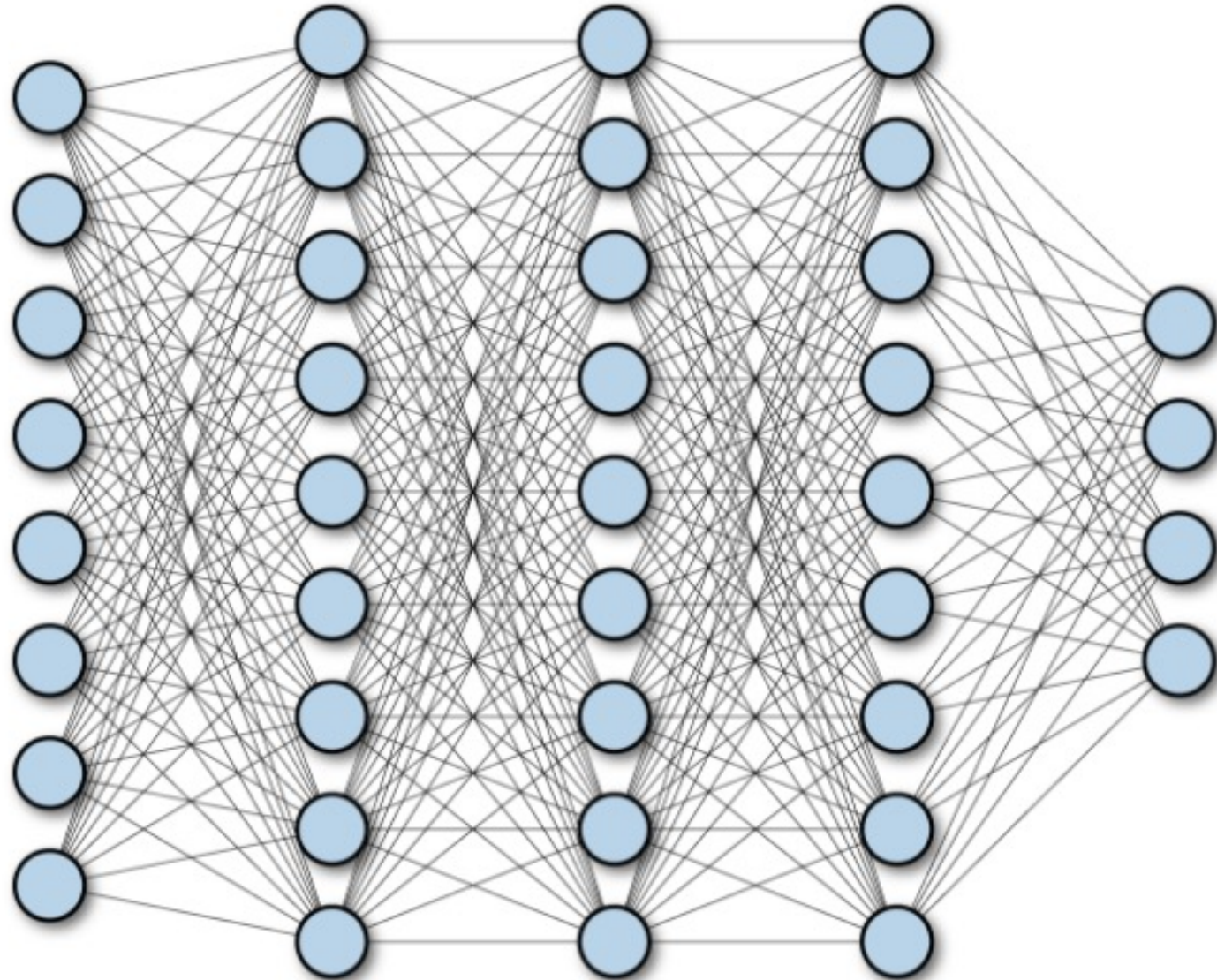
5.Flattening

- Converts multi-dimensional feature maps into a one-dimensional vector.
- Prepares the data for the fully connected layer.
- Used before classification
- Example:
 - Feature map size: **$16 \times 16 \times 12$**
 - Flattened vector size: **3072**



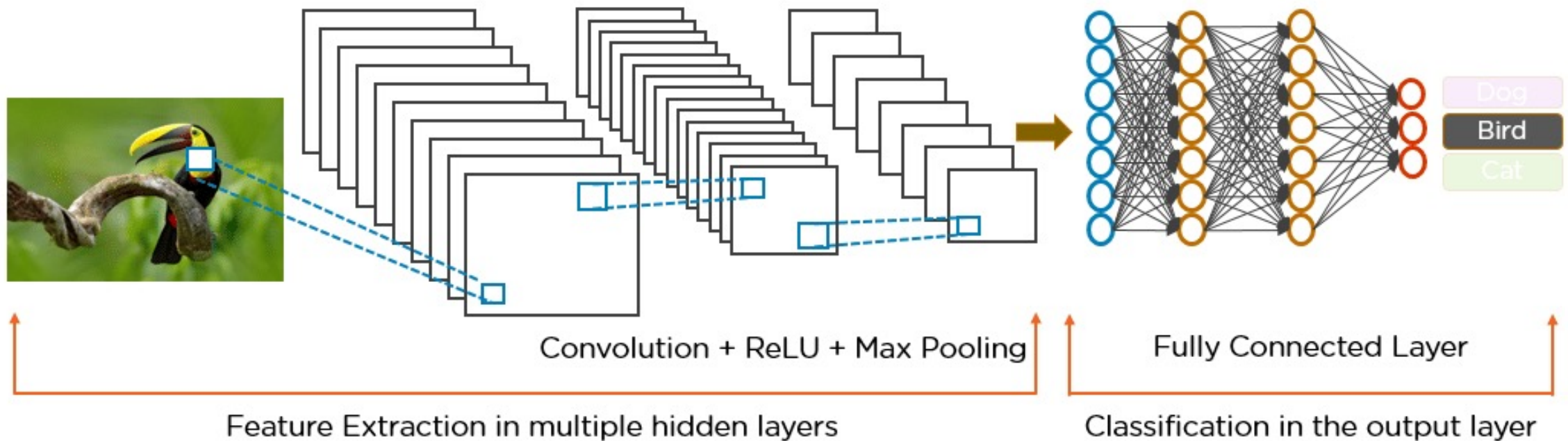
6. Fully Connected Layer

- Performs high-level reasoning using extracted features.
- Every neuron is connected to all neurons in the next layer.
- Produces the final classification scores.



7. Output Layer

- Produces the final prediction of the network.
- Converts scores into probabilities.
- Common activation functions:
 - **Sigmoid** → binary classification
 - **Softmax** → multi-class classification
- Outputs the predicted class or probability values.



CNN Propagation

